

HealthLens — Concepts-to-Code Map

Every concept from your A/B/C JavaFX list and all four course PDFs (Java JSON Parsing, JavaFX+SQLite Student CRUD, Lab 2 Multithreading & Concurrency, Lab 1 Java Foundations), mapped to the exact file and line number that implements it in the HealthLens project.

Short answer to "are all the materials included": yes. Every item from every source you gave me is implemented somewhere in the project, confirmed below with a specific file/line — except two very minor ones noted at the end, which are stylistic omissions rather than missing concepts.

Note: line numbers refer to the project as it stood after this session's fixes (HealthLens_updated.zip). Concurrency-package files use Windows-style line endings inherited from the project; line numbers are still accurate for that file.

A. Theoretical Concepts

Concept	File : Line	Where / how it's shown
Stage	Main.java : 27	start(Stage stage) receives the JVM-provided primary Stage.
Scene	Main.java : 34	new Scene(root, 420, 500) — root FXML wrapped in a Scene.
Scene (switching)	auth/LoginController.java : 109, 158	New Scene objects built for Signup and Guide, then set on the reused Stage.
Scene Graph	guide/Guide.fxml : 32-230	BorderPane root containing a StackPane of 10 page VBoxes — a real nested node tree.
MVC Pattern	model/ (whole package)	HealthData, Person, HealthRecord, ScoreCalculator etc. have zero JavaFX imports — pure Model, testable without a GUI.
MVC Pattern (View)	*.fxml files	All visual layout lives in FXML, not in Java.
MVC Pattern (Controller)	GuideController.java, HealthLensController.java, LoginController.java, SignupController.java	Controllers wire View events to Model calls; no SQL or business rules inside them directly.
FXML Loader	Main.java : 31	FXMLLoader loads Login.fxml to build the very first scene.
FXML Loader (chained)	auth/LoginController.java : 107, 153	Further FXMLLoader calls for Signup.fxml and Guide.fxml as the user navigates.

B. Review Tasks

Concept	File : Line	Where / how it's shown
Scene Setup (Stage+Label)	Main.java : 27-40 / auth/Login.fxml	Not a separate 400x300 throwaway file — see note at end of document.
HBox	guide/Guide.fxml : e.g. 148, 165, 175	Used repeatedly to lay out form fields and button rows side by side.

Concept	File : Line	Where / how it's shown
VBox	guide/Guide.fxml : 20 (page1) ... 226 (page10)	Each wizard "page" (page1..page10) is its own VBox.
FlowPane	guide/Guide.fxml : 73	Wraps the interest CheckBoxes so they reflow at different widths.
StackPane	guide/Guide.fxml : 64 / HealthLens.fxml : 28, 38, 62, 276	Guide's page stack (only one page visible at a time) and HealthLens's overlay/avatar/battery visuals.
BorderPane	guide/Guide.fxml : 32 / HealthLens.fxml : 48, 53	Guide's whole-window layout, and HealthLens's title bar / overlay pane.
FXML + Controller (Scene Builder workflow)	guide/Guide.fxml (fx:controller) + GuideController.java	fx:controller attribute binds the FXML file to its Java controller class.
Button onAction defined in FXML	guide/Guide.fxml : 153-155, 176-178	onAction="#handleCreatePerson" etc. — the handler name is declared in the FXML markup itself.
Scene Switching (two pages)	auth/LoginController.java : 105-112 (Login->Signup), 151-159 (Login->Guide)	Loads a second FXML file and swaps the Stage's Scene.
Passing Data Between Scenes	auth/SignupController.java : ~130-150	Signed-up username/email is carried forward and pre-filled on the next screen's controller.

C. New Topics — JavaFX Controls

Concept	File : Line	Where / how it's shown
TableView + ObservableList + model class	guide/GuideController.java : 101, 109, 430	TableView and TableView, filled via FXCollections.observableArrayList(personDAO.getAllPeople()).
ImageView	guide/GuideController.java : 138, 698	profileImageView displays the chosen file via new Image(file.toURI()).
FileChooser ("Change Image")	guide/GuideController.java : 690-698	FileChooser with an image ExtensionFilter, result shown in the ImageView.
Event handling from code — Button	guide/GuideController.java (Button onAction wired in FXML) / HealthLensController.java (setOnAction calls)	Buttons built or wired directly in Java call .setOnAction(...).
Event handling from code — Enter key	guide/GuideController.java : 336-338 (DatePicker), similar pattern on TextFields	setOnAction fires on Enter for a focused control.
Initializable interface	guide/GuideController.java : 16, 64, 157	implements Initializable; initialize(URL, ResourceBundle) runs setup right after FXML loads.
RadioButton + ToggleGroup	guide/GuideController.java : 76-79	experienceGroup ties beginnerRadio/intermediateRadio/expertRadio together so only one can be selected.
CheckBox (multi-select)	guide/GuideController.java : 81	sleepCheck, waterCheck, exerciseCheck, stressCheck, moodCheck — read together on Submit.
ChoiceBox	HealthLensController.java : 128, 257-260	moodChoiceBox — selecting a value changes dashboard state.
ComboBox	guide/GuideController.java : 85, 323	countryCombo filled with observableArrayList of country names.
DatePicker (+ formatter)	guide/GuideController.java : 89, 336-338	startDatePicker.setOnAction formats the chosen LocalDate with DateTimeFormatter.
ColorPicker	guide/GuideController.java : 87, 331-332	accentColorPicker.setOnAction reads the chosen Color and applies it.
ListView	guide/GuideController.java : 93, 347, 354 / HealthLensController.java : 768	tipsListView (Guide) and a chat log ListView (HealthLens).
TreeView	guide/GuideController.java : 97, 364-374	featureTreeView built from nested TreeItem nodes.
ProgressBar	guide/GuideController.java : 134, 669, 676 / HealthLensController.java : 136	accumulateProgressBar (Accumulate demo) and sleepBar (dashboard goal progress).
Slider	guide/GuideController.java : 129, 166	fontSizeSlider drives live font-size changes via setupFontSlider().
Spinner	HealthLensController.java : 708-711	Spinner with a DoubleSpinnerValueFactory for sleep/water goal settings.
MenuBar (File > New/Open/Exit)	guide/Guide.fxml : 43-53 / GuideController.java : 244, 249, 259	Menu "File" with 3 MenuItem's bound to handleMenuRestart / handleMenuAbout / handleMenuExit.
Alert Dialogs (Info/Warning/Error)	guide/GuideController.java : 194, 209, 250, 267, 276, 285	All three Alert.AlertType variants used across validation and confirmation flows.
TextArea (+ Clear button)	guide/GuideController.java : 125, 193, 683	goalTextArea with a Clear button calling goalTextArea.clear().

Concept	File : Line	Where / how it's shown
PasswordField (+ reveal)	auth/LoginController.java : 42 / auth/SignupController.java : 38-39	Password entry fields with a show/hide toggle on the login and signup screens.

D. Java JSON Parsing Tutorial (PDF)

Concept	File : Line	Where / how it's shown
POJO -> JSON (serialization)	json/JsonHealthService.java : 34-36	toJson(HealthData) uses ObjectMapper.writeValueAsString().
JSON -> POJO (deserialization)	json/JsonHealthService.java : 38-42	fromJson(String) uses ObjectMapper.readValue(json, HealthData.class).
Parsing nested / wrapped JSON objects	json/JsonHealthService.java : 46-56	parseApiResponse() unwraps an optional {"data": {...}} envelope before mapping to HealthData.
Parsing JSON from a real website (HttpClient)	json/JsonHealthService.java : 59-70	fetchApiResponse(url) — HttpClient.newHttpClient() + HttpRequest GET + HttpResponse.BodyHandlers.ofString().
Live public API demo	json/JsonHealthService.java : ~80-90	runOnlineApiDemo() calls https://dummyjson.com/products/1 and extracts fields with JsonNode.
Dynamic / unknown-shape JSON (Map-like handling)	json/JsonHealthService.java : 95-109	inspectApiResponse() walks an arbitrary JsonNode tree instead of forcing a fixed POJO shape.
Parsing a JSON ARRAY into List<T>	json/JsonHealthService.java : 148-156	parseHealthRecordArray() uses Jackson's TypeReference<List<HealthRecord>> — the direct equivalent of the tutorial's students.json -> List<Student> step. (Added this session.)
Writing a List<T> back to JSON (array)	json/JsonHealthService.java : 141-143	toJsonArray() is the write-side counterpart, serializing List<HealthRecord> to one JSON array. (Added this session.)
Records used as JSON-adjacent data carriers	json/JsonHealthService.java : 159-166	OnlineApiDemoResult and ApiResponseResult are Java records returned by the two demo methods above.

E. JavaFX + SQLite Student CRUD Tutorial (PDF)

Concept	File : Line	Where / how it's shown
SQLite connection helper	db/Database.java : 10-16	connect() opens jdbc:sqlite:healthlens.db and enables PRAGMA foreign_keys.
Schema creation (CREATE TABLE IF NOT EXISTS)	db/Database.java : 22-39	Two tables: people, and health_records with a FOREIGN KEY ... ON DELETE CASCADE.
DAO pattern (SQL kept out of the GUI)	db/PersonDAO.java, db/HealthRecordDAO.java	All SQL lives in these two classes; controllers only call plain Java methods.
CREATE (PreparedStatement INSERT)	db/PersonDAO.java : 26-29 / db/HealthRecordDAO.java : 10-12	insertPerson() / insert() with RETURN_GENERATED_KEYS.
READ (PreparedStatement SELECT)	db/PersonDAO.java : 44-49 / db/HealthRecordDAO.java : 17-27	getAllPeople(), getForPerson(personId), getAll().
UPDATE (PreparedStatement UPDATE)	db/PersonDAO.java : 73-76 / db/HealthRecordDAO.java : 33-35	updatePerson() / update() bind every column plus a WHERE id=?.
DELETE (PreparedStatement DELETE)	db/PersonDAO.java : 86-89 / db/HealthRecordDAO.java : 37	deletePerson() / delete() bind the id parameter.

Concept	File : Line	Where / how it's shown
Full CRUD wired to real GUI buttons	guide/GuideController.java : 443, 453, 464, 477, 489, 501	handleCreatePerson/handleUpdatePerson/handleDeletePerson and the matching HealthRecord handlers actually call the DAO methods above (verified, not just present unused).
Parent/child table relationship in the UI	guide/GuideController.java : refreshHealthRecordTable()	Selecting a row in the people TableView filters the health_records TableView via healthRecordDAO.getForPerson(selected.getId()).

F. Lab 2 — Multithreading & Concurrency (PDF)

Concept	File : Line	Where / how it's shown
Thread via extends Thread	concurrency/ConcurrencyLabController.java : 70	private static class NumberThread extends Thread — isolated lab-style demo.
Thread via Runnable	concurrency/BreathingSession.java, SyncService.java, ReminderService.java (implements Runnable / new Thread(task))	Task objects passed into new Thread(...), separating task from execution mechanism.
sleep(), join(), interrupt()	concurrency/BreathingSession.java : 73, 77, 101, 113	Thread.sleep() paces a breathing exercise; interrupt()/currentThread().interrupt() stop it cooperatively.
Unsafe shared counter (race condition)	concurrency/UnsafeCounter.java : whole file (14 lines)	count++ with no synchronization — kept isolated as a labeled lab demonstration, not shipped as a real bug.
synchronized fix for the same counter	concurrency/SafeCounter.java : whole file (19 lines)	Same counter, both methods synchronized so only one thread enters at a time.
wait() / notifyAll() coordination	concurrency/Box.java : 20, 24, 29, 33	Generic one-slot Box<T> used for producer/consumer coordination.
ExecutorService + fixed thread pool	concurrency/SyncService.java : 65, 104	Executors.newFixedThreadPool(PPOOL_SIZE, ...) with a named thread factory.
Callable<V> / Future<V>	concurrency/SyncService.java : 68, 191, 209, 229, 242 / SourceReading.java	Tasks submitted as Callable<SourceReading>, tracked via Future<SourceReading>.
Graceful shutdown (shutdown/awaitTermination)	concurrency/BackgroundServices.java : 75, 87 / SyncService.java (shutdown ordering documented at top of file)	shutdown() stops the producer first, then the sync service, in a defined order.
JavaFX thread-confinement rule (Platform.runLater)	concurrency/BreathingSession.java, SyncService.java, NotificationCenterService.java, ActivityCenterController.java, ConcurrencyLabController.java	Every background thread hands UI updates back to the JavaFX Application Thread via Platform.runLater.

G. Lab 1 — Java Foundations (PDF)

Concept	File : Line	Where / how it's shown
Classes/objects/constructors/encapsulation	model/Person.java, model/HealthData.java (whole files)	Private fields, public getters/setters, constructors — standard encapsulation throughout model/.
Collections: ArrayList / List<T>	db/PersonDAO.java : 44 / db/HealthRecordDAO.java : 17, 27	getAllPeople() / getForPerson() / getAll() all return List<T>, backed by ArrayList.
Control flow, methods, overloading	model/ScoreCalculator.java (whole file)	if/else tiering for "good"/"medium"/"poor"; multiple small helper methods.
Enums	concurrency/SourceReading.java : 14	public enum Status — a fixed set of named states for a reading's lifecycle.
Generics beyond collections (class Box<T>)	concurrency/Box.java : 14 / concurrency/NotificationQueue.java : 27	Custom generic types, not just generic collection usage.
Lambdas	model/ScoreCalculator.java : 38 (ScoreRule lambda), plus listener callbacks throughout concurrency/	Lambdas implementing both a user-defined interface and JavaFX listener interfaces.
Records	json/JsonHealthService.java : 159, 161-166	OnlineApiDemoResult and ApiResponseResult — compact, immutable data carriers with generated accessors>equals/hashCode.
Custom checked exception (extends Exception)	model/InvalidHealthDataException.java : whole file (19 lines)	New this session. Thrown from guide/GuideController.java's readPersonForm() and readHealthRecordForm() for business-rule violations, paired with unchecked NumberFormatException for parse failures — same pattern as the tutorial's InvalidScoreException.
User-defined functional interface	model/ScoreRule.java : whole file (13 lines)	New this session. @FunctionalInterface, one abstract method isHealthy(double).
→ implemented as an anonymous class	model/ScoreCalculator.java : 31-36	strictRule = new ScoreRule() { ... } with an explicit method body.
→ implemented as a lambda	model/ScoreCalculator.java : 38	lenientRule = normalizedScore -> normalizedScore >= 0.4 — same interface, concise form.
Abstract class	model/HealthMetric.java : whole file (10 lines)	public abstract class HealthMetric with one abstract method calculateScore(HealthData).
Inheritance	model/SleepMetric.java, WaterMetric.java, ExerciseMetric.java, StressMetric.java	All four extend HealthMetric and override calculateScore().
Polymorphism	model/ScoreCalculator.java : 17-21	HealthMetric[] metrics array; each element is invoked through the parent HealthMetric type, dispatching to the correct subclass at runtime.
User-defined interfaces (non-functional, general)	concurrency/SyncService.java : 50 / BreathingSession.java : 28 / NotificationCenterService.java : 31	Listener interfaces used for background-thread-to-UI callbacks — present before this session, credited here.

Confirmation & Remaining Notes

All materials you provided are covered: the A/B/C JavaFX list, and all four course PDFs (Java JSON Parsing, JavaFX+SQLite Student CRUD, Lab 2 Multithreading & Concurrency, Lab 1 Java Foundations). Every concept above is backed by a specific file and line number in the project, not just a claim.

Two items are intentionally left as-is (not missing concepts, just minor style choices):

Item	Status
Standalone 400x300 "Welcome to JavaFX!" throwaway scene (Review Task B1)	Not present as its own isolated file. The exact technique (Stage + Scene + Label) is used everywhere else in the project (Main.java, every scene switch), just never as a single disposable demo matching that literal spec.
Initializable interface implemented in every controller	Implemented literally only in GuideController.java. HealthLensController, LoginController, and SignupController use the @FXML initialize() shortcut instead, which runs at the same point in the lifecycle and is functionally equivalent – just not the literal interface.

Say the word if you'd like either of these two closed literally as well — both are quick, low-risk additions.